# An Offline Mobile-Based App for Visual Learning of Algorithms


Jaya Gupta
*Department of computer science and engineering (AIML)*
*A.P. Shah Institute of Technology*
Thane(W)-400615, India
jdgupta@apsit.edu.in

Bharti Khemani
*Department of computer science and engineering (AIML)*
*A.P. Shah Institute of Technology*
Thane(W)-400615, India
bjkhemani@apsit.edu.in

Ikra Sayyad
*Department of computer science and engineering (AIML)*
*A.P. Shah Institute of Technology*
Thane(W)-400615, India
ikrasayyad4@gmail.com

Shikshita Yadav
*Department of computer science and engineering (AIML)*
*A.P. Shah Institute of Technology*
Thane(W)-400615, India
shikshitayadav9502@gmail.com

Disha Waghmare
*Department of computer science and engineering (AIML)*
*A.P. Shah Institute of Technology*
Thane(W)-400615, India
dishawaghmare209@gmail.com

Pratiksha Pathak
*Department of computer science and engineering (AIML)*
*A.P. Shah Institute of Technology*
Thane(W)-400615, India
pratikshapathak491@gmail.com



***Abstract*— DSA and other difficult subjects are now easier to understand thanks to mobile-based learning. When they read books or attend lectures, many students barely understand how algorithms operate. This Android app uses animations to display the steps of any algorithm. By providing clear graphics, pseudocode, and controls to pause, replay, or alter the inputs, it aids users in understanding various sorting, tree, and graph algorithms. Because it is entirely offline and modular in design, students with limited resources will find it very helpful. Students learn DSA more easily and effectively because theoretical concepts are transformed into interactive, visual learning.**



***Keywords— data structures and algorithms, algorithm visualization, android app, mobile-based learning, offline education.***


## I. Introduction

The foundation of computer science education is Data Structures and Algorithms (DSA). However, these ideas' abstract nature frequently makes comprehension difficult. While algorithms function through dynamic sequences of operations—such as swapping elements, traversing nodes, or updating weights—that are challenging to mentally visualise, textbooks describe how data is arranged.

In order to overcome this, we created this app using Android Studio, the official Integrated Development Environment (IDE) for Android. This instructional mobile application uses animated visualisations to make algorithmic concepts easier to understand. Through real-time execution flow observation, the app enables students to investigate sorting strategies, tree traversals, and graph algorithms.

### *A. Need for the Application*

The majority of visualisation tools currently in use are web-based, necessitating desktop browsers and active internet connections, which restricts accessibility for students who are on the go. It fills the need for an interactive, offline, and portable learning tool that students can use at any time to get ready for competitive programming, technical interviews, or academics.

### *B. What is DSA and Algorithm Visualization?*

Algorithms are the sequential processes used to manipulate data, whereas data structures (such as arrays, trees, and graphs) are ways to store data. The graphical depiction of these processes is called algorithm visualisation. Visualisation enables the learner to observe the "flow" of logic—how comparisons are made, pointers


XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

move, and the data structure changes over time—instead of reading static code. Conceptual clarity and logical retention are greatly improved by this visual feedback loop.

### C. *Overview of Android Studio*

Android Studio is a one-stop solution for building Android applications. It provides a complete code editor, layout designer, and testing simulators. For this project, we utilized Android Studio to develop a modular Java-based application that is lightweight and supports multiple devices.

## II. LITERATURE SURVEY

We examined current solutions to find gaps in the market before creating this app.

i. Web-Based Solutions: Kumari et al. (2022) and Singh & Patel (2023) proposed web-based platforms for sorting and searching algorithms. While effective, these systems are dependent on continuous internet access and are not optimized for mobile interfaces[1][8]. Pierson and Rodger (1998) developed JAWAA, a web-based system for animating data structures[13]. Naps (2005) developed Jhavé, which assists in the visualization of algorithms in computer science education[14].

ii. Gamification & AI: Bhandari et al. (2025) introduced an AI chatbot integration for DSA learning. While engaging, it lacks a dedicated offline mobile framework[5].

iii. Desktop Applications: Legacy software exists for PCs, but as noted by Zhang (2024), there is a distinct lack of portable tools that allow for revision during commutes or in areas with poor connectivity[4].

iv. Previous Mobile Attempts: Kocakoyun (2019) discussed Android development for education, but few dedicated DSA apps exist that combine Sorting, Trees, and Graphs into a single offline package without intrusive advertisements[10].

v. Improved Algorithm Visualization Systems: Lanjewar et al. 2022 and Kogan et al. 2024 showed that animated and interactive visualizations improve understanding of data structures and algorithms. However, these methods are limited to web-based platforms and do not support offline, mobile-first learning [2][3].

vi. Visualization and Mobile Learning Approaches: Liu (2023) and Lou et al. (2023) proved that animated and interactive graphics can help people understand algorithms a lot [6][7]. Gupta and Rao (2023) emphasized that algorithm visualization is a useful educational tool but noted that there is no all-in-one learning platform available yet [9].

vii. Evaluation of algorithm visualisation: Kumari et al. examined software projects for visualizing algorithms [11].

TABLE 1 RESEARCH GAP

| Ref & Year | Algorithm | Research Gap |
|---|---|---|
| [1] 2022 | General Algorithm Animation | Limited to internet access alone; not mobile or offline compatible. |
| [2] 2022 | Sorting, Searching | Focuses on improvement without a modular approach in mobile design. |
| [3] 2024 | Sorting, Recursion, Graphs | Limited coverage of algorithms; no offline support. |
| [4] 2024 | Data Structures and Algorithms | Instruction geared more toward desktops; lacks mobile or offline access. |
| [5] 2025 | Sorting, Searching | Web-based, no offline capability, and no modular expansion. |
| [6] 2024 | General Algorithm Animation | Uses only web technologies; it does not have a mobile or modular extension. |
| [7] 2024 | Intelligent Algorithms and DS | Requires online connectivity; not focused on mobile or offline use. |
| [8] 2023 | Advanced Algorithms | Web-only; no mobile/offline features. |
| [9] 2023 | Sorting, Trees, Graphs | Survey only; no practical implementation |
| [10] 2019 | Android activity lifecycle & event-driven programming | Lack of practical, step-by-step, and scholarly guidelines for beginners. |

Identified Gap**:** As per described in Table 1, there is a need for a unified, offline mobile application that visualizes a wide breadth of algorithms (Sorting, Trees, Graphs) with synchronized pseudocode.

## III. Proposed System

This is an Android-based app designed to visualize the internal execution of algorithms.

### A. *Features*

- Algorithm Selection Panel**:** Categorized menu for Sorting, Trees, and Graphs.
- Input Customization**:** Supports both user-defined
- input and random data generation.
- Step-by-Step Visualization**:** Animations for swaps, traversals, and pointer movements.
- Pseudocode Highlighting**:** Highlights the specific line of code executing in sync with the animation.
- Interactive Controls**:** Play, Pause, Reset, and Speed Control (Slow/Fast).
- Offline Capability**:** Fully functional without internet connectivity.

### B. System *Workflow*

The workflow follows a linear user journey:

- Initialization: User selects a category (for example, Graphs).
- Selection: User selects an algorithm (for example, BFS).
- Input: User provides data or selects "Random".
- Processing**:** The logic layer calculates steps.
- Visualization: The UI layer renders the steps with pseudocode.

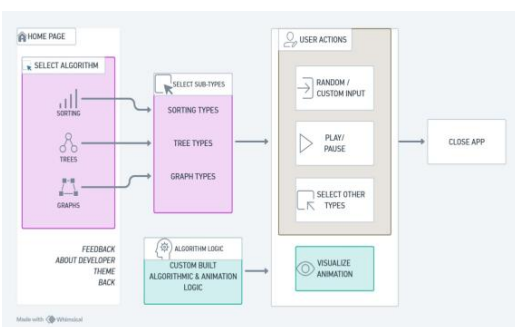


Fig. 1. Workflow

## IV. Implementation Methodology

i. Animation Engine: Custom View components were created to render nodes and bars. Transitions were handled using Android's ObjectAnimator and Handler classes to ensure smooth frame updates.

ii. Logic Modules: Each algorithm (for example, QuickSort.java) is encapsulated in a separate class to maintain modularity.

iii. Synchronization: A state-management system links the logical step (array index change) to the UI thread, ensuring the visual update happens exactly when the logic executes.

## V. Result

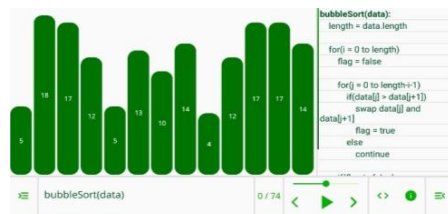


Fig. 2. Before Sorting

Fig 2 is an example of BubbleSort, which shows the initial state before execution. The vertical bars of different heights represent unsorted elements in an array.

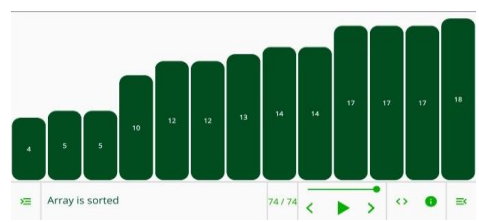


Fig. 3. After Sorting

Fig 3 shows the final state after completion of the sorting algorithm execution. Hence, the vertical bars are now sorted in ascending order.

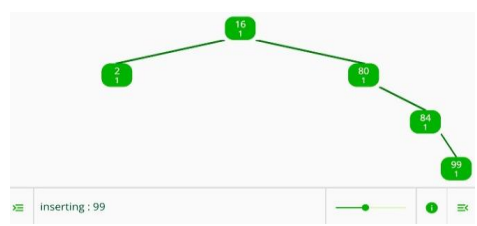


Fig. 4. Unbalanced AVL Tree

Fig 4 indicates the unbalanced AVL tree because the difference between the left sub-tree and the right sub-tree is not -1,0 or 1.

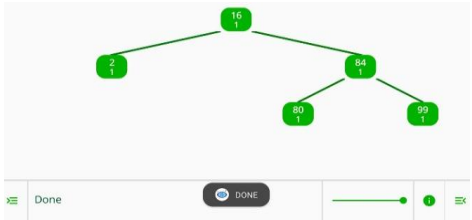


Fig. 5. Balanced AVL Tree

Fig 5 illustrates the balanced AVL tree after performing LL (left-left) rotation which rebalances the tree according to the property of AVL tree.

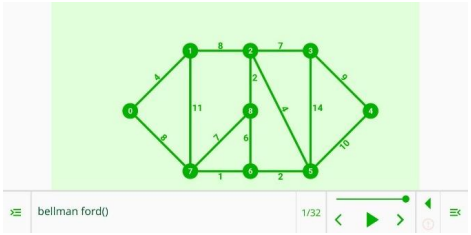


Fig. 6. Graph Visualization

Fig 6 indicates the weighted graph for BellmanFord algorithm, before finding the shortest path where all edges and vertices are in their initial state.

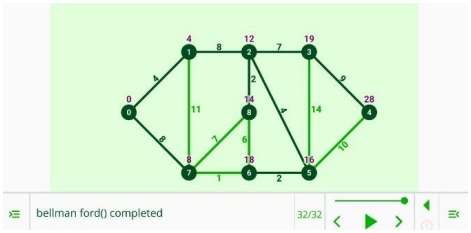


Fig. 7. Shortest Path

Fig 7 shows the highlighted path that is the shortest path from the source vertex to every other vertex in the graph which indicates the minimum cost path obtained after edge relaxation.

## VI. Conclusion

DSA is the basis for effective software engineering and problem-solving in computer science. However, students often find it hard to understand these ideas through traditional text-based materials and classroom lectures because algorithmic processes are always changing. This project addresses the gap between theoretical knowledge and practical understanding by providing a portable, offline, and interactive visualization platform. By taking advantage of Android Studio, the application presents step-by-step visual animations that demonstrate how complex algorithms operate internally. Users can observe data transformations in real time, control execution flow through pause and replay options, and experiment with different input values to strengthen conceptual clarity. This project highlights the transformative role of mobile technology in modern technical education by converting abstract DSA concepts into intuitive visual experiences.

## References

[1] Kumari, A. S. H. I. S. H., Mittal, M. A. N. A. V., Jha, V. I. P. U. L., Sahu, A. B. H. I. S. H. E. K., Kumar, M. A. N. I. S. H., Sangwan, N. E. E. T. I., & Bohra, N. A. V. D. E. E. P. (2022). Algorithm visualisation—modern web-based visualization of sorting and searching Appl Math Sci, 21.. algorithms. Adv Appl Math Sci, 21.

[2] Lanjewar, R., Mehra, N., Doshi, A., & Mane, Y. (2022). Algovis–An Enhanced Way to Visualize Algorithms. In ITM Web of Conferences (Vol. 44, p. 03013). EDP Sciences.

[3] Kogan, G., Chassidim, H., & Rabaev, I. (2024). The efficacy of animation and visualization in teaching data structures: a case study. Educational technology research and development, 72(4), 2349 2372.

[4] Zhang, Yan. (2024). Application of Algorithm Visualization Techniques in Teaching Computer Data Structure Course. Applied Mathematics and Nonlinear Sciences. 9. 10.2478/amns-2024-0788..

[5] Bhandari, S. R., Abhijeet, C., et al., 2025. Algorithm visualization on data structures along with AI chatbot. IJIRCST, 13(1), 21–27.

[6] Liu, G. (2023, July). Research and Application of Animation Visualization Based on HTML5 Algorithm. In International Conference on Frontier Computing (pp. 188-193). Singapore: Springer Nature Singapore.

[7] Lou, G., Sufang, W., & Shen, H. (2023, August). Research on Graphic Interactive Visual Online Teaching of Intelligent Algorithm Course. In EAI International Conference, BigIoT-EDU (pp. 506 516). Cham: Springer Nature Switzerland.

[8] Matalia, Jeet & Kathiriand, Shreya & Patel, Priyanka. (2024). Demystifying Complex Algorithms: A Visual Guide Through the DSA Visualizer. 10.1007/978-981-97-1313-4_18.

[9] Gupta, V., & Rao, D., 2023. Visualizing algorithms in the field of education: A comprehensive review. IJRASET, 11(7), 250–258.

[10] Kocakoyun, Ş. (2017). Developing of android mobile application using java and eclipse: an application. International Journal of Electronics Mechanical and Mechatronics Engineering, 7(1), 1335-1354.

[11] Kumari, A., Bohra, N., & Sangwan, P. Evaluating Software Algorithm Visualization Projects.

[12] Tuparov, G., Tuparova, D., & Jordanov, V. (2014). Teaching sorting and searching algorithms through simulation-based learning objects in an introductory programming course. Procedia-Social and Behavioral Sciences, 116, 2962-2966.

[13] Pierson, W. C., & Rodger, S. H. (1998). Web-based animation of data structures using JAWAA. ACM SIGCSE Bulletin, 30(1), 267-271.

[14] Naps, T. L. (2005). Jhavé: Supporting algorithm visualization. IEEE Computer Graphics and Applications, 25(5), 49-55.

[15] Karavirta, V., & Shaffer, C. A. (2015). Creating engaging online learning material with the jsav javascript algorithm visualization library. *IEEE Transactions on Learning Technologies*, *9*(2), 171-183.